

HANDS-ON 3: Test Automation Process, Lifecycle & Framework Types Task 1: Automation Decision and Test Case Selection

1.1 Automation Criteria Criteria 1: Repetitiveness

text Definition: Tests that are executed frequently (daily, weekly, or on every build) Application to Course Management API: The POST /api/courses/ endpoint is called in every CI/CD pipeline, every regression suite, and every deployment Decision: HIGH PRIORITY for automation ROI: Manual testing of this endpoint 10 times per week * 10 minutes = 100 minutes/week wasted Criteria 2: Risk

text Definition: High business impact if the feature breaks Application to Course Management API: Course creation is a core business function. If broken, students cannot enroll, causing revenue loss and user frustration Decision: HIGH PRIORITY for automation Risk Level: Critical (P1) Criteria 3: Time Required for Manual Testing

text Definition: Time-consuming to test manually with all data combinations Application to Course Management API: Testing 50 data combinations (valid, invalid, edge cases) manually would take hours Decision: HIGH PRIORITY for automation Time Saved: 4 hours of manual testing reduced to 5 minutes of automated testing Criteria 4: Data-Driven Nature

text Definition: Same test needs to be executed with different data sets Application to Course Management API: Course creation needs to test multiple scenarios: valid data, missing fields, duplicate codes, special characters, long names, etc. Decision: HIGH PRIORITY for automation Data Sets: 20+ different test data combinations Criteria 5: Stability

text Definition: Feature is stable and unlikely to change frequently Application to Course Management API: API contract is stable and approved. No major changes expected for next 6 months Decision: HIGH PRIORITY for automation Stability Score: 8/10

1.2 Manual vs Automation Decisions

Test Case Description	Decision	Justification
(a) Regression test for all CRUD endpoints after every code change (core functionality), many test cases (20+), time-consuming testing	AUTOMATE	Repetitive (every build), high-risk endpoints (core functionality), many test cases (20+), time-consuming (hours of manual testing)
(b) Exploratory testing of a new search feature. The feature is rapidly changing, unpredictable user behavior, automation would be wasted effort	MANUAL	Requires human judgment and creativity, search feature is rapidly changing, unpredictable user behavior, automation would be wasted effort
(c) Performance test: 100 concurrent users calling GET /api/courses/ consistently repeat, needs to be run in CI/CD, requires specialized tools	AUTOMATE	Can't simulate 100 users manually, users calling GET /api/courses/ consistently repeat, needs to be run in CI/CD, requires specialized tools
(d) UI test for the login form. Repeated (every login), high-risk (critical functionality), stable UI, easy to automate with Selenium	AUTOMATE	Repeated (every login), high-risk (critical functionality), stable UI, easy to automate with Selenium
(e) Verify the API documentation changes are reviewed by humans, automation would be expensive	MANUAL	Requires human judgment, one-time check, (Swagger) is accurate, documentation changes are reviewed by humans, automation would be expensive
(f) Smoke test: verify the API is reachable after deployment, repeated after every deployment, low maintenance, catches critical issues early	AUTOMATE	Fast to automate (single API call), reachable after deployment, repeated after every deployment, low maintenance, catches critical issues early

1.3 Test Automation ROI Calculation Given:

text Automation time: 4 hours = 240 minutes (initial setup) Manual execution time: 30 minutes per run Maintenance overhead: 20% of manual time (6 minutes) per run after the 10th run Calculation:

text Manual cost per run = 30 minutes Automation cost per run = 4 hours = 240 minutes (initial) + 6 minutes maintenance per run

Break-even point: Let n = number of runs Manual total time = $30n$ Automation total time = $240 + (6 * (n - 10))$
[after 10 runs, 20% of 30 = 6 min per run]

$30n = 240 + 6(n - 10)$ [for $n > 10$] $30n = 240 + 6n - 60$ $30n - 6n = 180$ $24n = 180$ $n = 7.5$

Answer: Automation pays for itself after 8 runs Visual Representation:

Runs	Manual Time (min)	Automation Time (min)	Difference
1	30	240	-210 (Loss)
5	150	240	-90 (Loss)
8	240	240	0 (Break-even)
10	300	240	60 (Gain)
20	600	300	300 (Gain)
50	1500	480	1020 (Gain)

Calculation: ROI

text After 50 runs: Automation saves 1020 minutes = 17 hours Annual savings: If run 50 times/year, saves 17 hours/year Monetary value: 17 hours * \$30/hour = \$510/year

1.4 Flaky Tests Definition:

text Tests that pass and fail inconsistently without any code changes. A flaky test undermines confidence in the entire test suite. Example of a Flaky Test:

python

Flaky test example

```
def test_course_creation(): driver.get("/create-course") driver.find_element(By.ID, "name").send_keys("Data Structures") driver.find_element(By.ID, "submit").click() # Sometimes fails because the success message takes longer to load assert "success" in driver.find_element(By.CLASS_NAME, "message").text
```

Strategies to Prevent or Fix Flaky Tests:

text Strategy 1: Use Explicit Waits Implementation: Replace `time.sleep(2)` with `WebDriverWait(driver, 10).until(EC.visibility_of_element_located((By.ID, "message")))`

Strategy 2: Use Stable Locators Implementation: Use unique IDs instead of dynamic classes or XPaths that change frequently

Strategy 3: Environment Isolation Implementation: Use test data isolation (each test creates its own data), reset database state between tests

Strategy 4: Retry Mechanism Implementation: Use pytest's `@pytest.mark.flaky(reruns=2)` decorator for known flaky tests

Strategy 5: Avoid Shared State Implementation: Don't share test data between tests; each test should be independent

Strategy 6: Handle AJAX Calls Implementation: Wait for AJAX calls to complete before interacting with elements

Task 2: Compare Automation Framework Types

2.1 Framework Type Comparison

Framework 1: Linear Framework

text Description: Simple script-based approach with no reusability. Each test is a separate script with hard-coded data and actions.

Advantage: Easy to create for beginners, no complex setup required, quick to prototype

Disadvantage: No code reuse, high maintenance, changes affect multiple scripts, difficult to scale

When to Use: Simple, one-time scripts or proof-of-concept projects

Example: test_course_creation_v1.py, test_course_creation_v2.py (separate files for each test) Framework 2: Modular Framework

text Description: Tests are divided into modules/functions for reusability. Common actions are extracted into functions.

Advantage: Reduced code duplication, easier to update, better organization

Disadvantage: Still has data hardcoded, requires programming knowledge, functions can become complex

When to Use: Medium complexity projects with stable applications

Example: def login(), def create_course(), def logout() functions reused across tests Framework 3: Data-Driven Framework

text Description: Test data is stored externally (Excel, CSV, JSON, databases). The same test logic runs with different data sets.

Advantage: Test multiple data combinations easily, data can be modified without changing code

Disadvantage: Complex to set up, requires data file management, no UI abstraction

When to Use: Heavy data variation, testing with 50+ data combinations

Example: test_data.xlsx with 50 rows of course data, one test reads and executes for each row Framework 4: Keyword-Driven Framework

text Description: Business-readable keywords represent actions (e.g., "Click Login", "Enter Text"). Non-technical team members can write tests.

Advantage: Non-technical team members can write tests, business-focused, readable

Disadvantage: Complex framework to build, performance overhead, requires keyword maintenance

When to Use: Business users writing tests, acceptance testing, BDD approach

Example: Click "Login", Enter "admin" in "Username", Verify "Welcome" message Framework 5: Hybrid Framework

text Description: Combines modular, data-driven, and/or keyword-driven approaches. Most common in real projects.

Advantage: Best of all worlds: reusability + data parameterization + readability

Disadvantage: Complex initial setup, requires skilled team members, steep learning curve

When to Use: Large projects, enterprise applications, teams with mixed technical skills

Example: Page Object Model (modular) + External data files (data-driven) + Gherkin/BDD (keyword-driven) 2.2 Framework Recommendation Scenario:

text The team is building a Selenium suite for the Course Management frontend. They need to:

- Test login with 50 different user/password combinations
- Reuse login steps across 20 test cases
- Support both technical and non-technical team members writing tests Recommended Framework: Hybrid (Modular + Data-Driven + Keyword-Driven)

Justification:

```

text +-----+ | Requirement | How Hybrid
Framework Meets It | +-----+ | Test login
with 50 different | Data-Driven: Login data stored in CSV/ | combinations | Excel. One test reads all 50 | |
combinations and executes for each | +-----+
+ | Reuse login steps across 20 | Modular: Login is a reusable function/ | | test cases | Page Object. All 20 test
cases call the | | same login function | +-----+
-+ | Support technical and non- | Keyword-Driven: Technical members write | | technical members | the code.
Non-technical members use | | Gherkin/BDD to write test scenarios | +-----+
-----+ Folder Structure:

```

```

text CourseManagementAutomation/ | +-- data/ | +-- login_data.csv (Data-driven: 50 user/password
combinations) | +-- course_data.json | +-- pages/ | +-- login_page.py (Modular: Reusable login functions) | +--
courses_page.py | +-- base_page.py | +-- features/ | +-- login.feature (Keyword-driven: Gherkin scenarios) | +--
- courses.feature | +-- step_definitions/ | +-- login_steps.py (Keywords implementation) | +-- courses_steps.py
| +-- tests/ | +-- test_login.py (Calls login_page functions) | +-- test_courses.py | +-- utils/ +-- data_loader.py
+-- driver_factory.py 2.3 Hybrid Framework Folder Structure text CourseManagementAutomation/ | +--
config/ | +-- config.yaml # Configuration settings | +-- environment.yaml # Environment-specific settings | +--
data/ | +-- login_data.csv # Data-driven test data | +-- course_data.json # JSON test data | +-- test_users.yaml
# YAML test data | +-- invalid_course_data.csv | +-- pages/ | +-- init.py | +-- base_page.py # Base class with
common methods | +-- login_page.py # Login page object | +-- courses_page.py # Course management page
| +-- course_detail_page.py # Course detail page | +-- profile_page.py # User profile page | +-- features/ | +--
login.feature # Gherkin scenarios for login | +-- course_creation.feature # Gherkin scenarios for courses | +--
course_enrollment.feature | +-- step_definitions/ | +-- init.py | +-- login_steps.py # Implementation of login
steps | +-- course_steps.py # Implementation of course steps | +-- common_steps.py # Shared step
implementations | +-- utils/ | +-- init.py | +-- driver_factory.py # WebDriver creation | +-- data_loader.py #
Data loading from files | +-- logger.py # Logging configuration | +-- screenshot.py # Screenshot utilities | +--
wait_utils.py # Custom wait functions | +-- fixtures/ | +-- init.py | +-- test_data.py # Test data fixtures | +--
environment.py # Environment fixtures | +-- tests/ | +-- init.py | +-- test_login.py # Login test cases | +--
test_courses.py # Course test cases | +-- test_enrollment.py # Enrollment test cases | +-- test_smoke.py #
Smoke test suite | +-- reports/ | +-- (Generated reports) | +-- screenshots/ | +-- (Screenshots on failure) | +--
logs/ | +-- test_execution.log | +-- requirements.txt # Python dependencies +-- pytest.ini # pytest
configuration +-- conftest.py # pytest fixtures +-- .gitignore # Git ignore file +-- README.md # Project
documentation +-- run_tests.py # Test execution script Key Takeaways text

```

1. Automate: Repetitive, high-risk, data-driven, stable tests

2. Manual: Exploratory, one-time, user experience, visual tests
3. ROI: Automation pays for itself after ~8 runs
4. Flaky Tests: Use waits, stable locators, isolation, retries
5. Framework Types: Linear, Modular, Data-Driven, Keyword-Driven, Hybrid
6. Hybrid: Most common in production projects
7. POM: Separates test logic from UI interaction
8. Data-Driven: External data files for parameterized testing
9. Keyword-Driven: Business-readable tests for non-technical team members